

Contents

Introduction	5
Architecture	7
Data Flow	11
Design Patterns	15
Architecture Decisions	21
Development	25
Database Schema	29

Introduction

About OutOf5K

OUTOF5K is a desktop application designed to help Counter-Strike 2 players analyze their gameplay, track improvement over time, and learn from professional players. The name refers to the aspiration of becoming one of the top 5,000 players.

Vision

The application aims to provide:

- Comprehensive demo file analysis
- Detailed statistics and performance tracking
- Visual replay and positioning analysis
- Comparison with professional players
- AI-powered coaching insights (future)

Target Users

OutOf5K is designed for:

- Competitive CS2 players looking to improve
- Teams wanting to review scrims and matches
- Coaches analyzing player performance
- Content creators reviewing highlights

Document Overview

This documentation provides comprehensive technical details for developing, understanding, and contributing to OutOf5K. It is organized as follows:

Chapter 2: Architecture System design, components, and their interactions

Chapter 3: Data Flow How data moves through the system

Figure 1: Primary user persona: Competitive CS2 player seeking improvement

Chapter 4: Patterns Design patterns used in the codebase

Chapter 5: Decisions Architecture Decision Records (ADRs)

Chapter 6: Development Setup, workflow, and contribution guidelines

Technology Stack

Layer	Technology	Purpose
Desktop Shell	Tauri v2	Native app container
Frontend	Svelte 5 + TypeScript	User interface
Styling	TailwindCSS	UI styling
Visualization	Pixi.js	2D map rendering
Backend	Rust	System integration
Demo Parser	Python + demoparser2	CS2 demo parsing
Database	SQLite	Local storage

Table 1: Technology stack summary

Quick Links

For specific topics, refer to:

- Getting started: Chapter
- System architecture: Chapter
- Database schema: Appendix

Architecture

Design Philosophy

OUTOF5K follows several core architectural principles that guide its design:

Local-First All demo processing happens locally on the user's machine, eliminating server costs and preserving user privacy.

Modular Architecture The system is composed of loosely coupled components that communicate through well-defined interfaces.

Progressive Enhancement Start with essential features and progressively add advanced capabilities.

Performance-Conscious CS2 demo files can be large (100MB+). The architecture handles this through background processing and incremental parsing.

System Context

OutOf5K operates as a standalone desktop application that interacts with several external systems:

- **Steam Platform:** User authentication and match history
- **HLSV:** Professional player statistics and match data
- **Faceit/ESEA:** Competitive match data and rankings
- **CS2 Game:** Demo file source

Container Architecture

The application is composed of four main containers, as shown in Figure 3.

Figure 2: C4 Context Diagram showing OutOf5K in relation to external systems. The C4 model provides a hierarchical way to describe software architecture at different levels of abstraction.

Figure 3: C4 Container Diagram showing the major components

Frontend (Svelte + TypeScript)

The user interface layer built with Svelte 5 and TypeScript. Key responsibilities:

- Rendering UI components
- User interaction handling
- 2D map visualization via Pixi.js
- Statistics charts and graphs
- State management via Svelte stores

Tauri Backend (Rust)

The desktop application shell and system integration layer:

- Native window management
- File system access for demo files
- SQLite database operations
- Python subprocess management
- IPC command handlers

Python Service

A subprocess handling computationally intensive demo parsing:

- CS2 demo file parsing via demoparser2
- Data extraction and transformation
- Heavy computational analysis
- Future: ML model inference

SQLite Database

Local persistent storage:

- Parsed demo metadata and events
- Player statistics and profiles
- User preferences and settings
- Cached external API data

Figure 4: Frontend component diagram

Component Details

Frontend Components

The frontend is organized into several key components:

Router Handles page navigation using SvelteKit’s routing

Dashboard Main landing page with activity feed

Demo Library Lists all imported demos with filtering

Demo Viewer Main analysis view with timeline and map

Map Canvas Pixi.js-based 2D rendering

Stores Svelte stores for state management

API Layer Typed wrappers around Tauri IPC

Rust Backend Components

Key backend components:

Commands Tauri IPC entry points

Services Business logic orchestration

Repositories Data access layer

Python Bridge Subprocess communication

API Clients External service integration

Python Service Components

The Python service includes:

Main Loop Command dispatch and response handling

Demo Parser demoparser2 integration

Event Processor Game event classification

Stats Calculator Performance metrics

Highlight Detector Notable moment detection

Technology Rationale

Detailed rationale for each decision is documented in the Architecture Decision Records (Chapter).

Figure 5: Rust backend component diagram

Figure 6: Python service component diagram

Decision	Rationale
Tauri	Small bundles, native performance
Svelte	Minimal boilerplate, reactive
Python subprocess	Best demo parser available
SQLite	Zero-config, file-based
Pixi.js	High-performance 2D canvas

Table 2: Key technology decisions

Data Flow

Overview

DATA FLOWS THROUGH OutOf5K in several key pathways:

1. Demo import and parsing
2. Statistics calculation and storage
3. Real-time playback visualization
4. External API synchronization

Demo Import Pipeline

The demo import pipeline transforms raw CS2 demo files into structured, queryable data.

Figure 7: Demo import sequence diagram

Pipeline Stages

1. **Import:** User selects demo files
2. **Validate:** Check file validity and format
3. **Parse:** Extract data via Python service
4. **Store:** Save to SQLite database

Parsing Process

The Python service extracts several categories of data:

Metadata Map, date, server, duration

Events Kills, damage, grenades, bomb actions

Positions Player locations over time

Statistics Per-player performance metrics

Highlights Aces, clutches, multi-kills

Phase	Progress
Header	0–10%
Events	10–60%
Positions	60–80%
Statistics	80–95%
Highlights	95–100%

Table 3: Parsing phases

IPC Protocol

Communication between Tauri and Python uses JSON over stdin/stdout.

Message Format

Request (Tauri → Python):

```
1 {  
2   "id": "uuid-string",  
3   "cmd": "parse",  
4   "params": {  
5     "file_path": "/path/to/demo.dem"  
6   }  
7 }
```

Response (Python → Tauri):

```
1 {  
2   "id": "uuid-string",  
3   "status": "success",  
4   "data": { ... }  
5 }
```

Progress Updates

During long-running operations, the Python service sends progress updates:

```
1 {  
2   "id": "uuid-string",  
3   "status": "progress",  
4   "progress": 50,  
5   "message": "Processing events..."  
6 }
```

Event System

Game Events

Game events extracted from demos follow a hierarchical model:

- **CombatEvent:** Kills, damage, blinds
- **WeaponEvent:** Fire, reload
- **GrenadeEvent:** Throw, detonate
- **BombEvent:** Plant, defuse, explode
- **RoundEvent:** Start, end, freeze

Application Events

The Tauri event system propagates updates to the frontend:

```
1 app.emit_all("demo:progress", DemoProgressPayload {
2     demo_id: id,
3     progress: 50,
4     message: "Parsing events...",
5 });
```

Playback Data Flow

Figure 8: Playback sequence diagram

During playback, data flows as follows:

1. User scrubs timeline
2. Store updates current tick
3. Map canvas queries positions at tick
4. Canvas renders player sprites
5. Event indicators update

External API Sync

Figure 9: External API synchronization flow

External data is fetched and cached:

1. Check cache freshness (24h TTL)
2. If stale, fetch from external API
3. Parse and transform to internal format
4. Store in SQLite cache table
5. Return to frontend

Query Patterns

Common database queries are optimized with indexes:

```
1 -- Get all kills for a player
2 SELECT * FROM events
3 WHERE demo_id = ?
4     AND event_type = 'KILL'
5     AND JSON_EXTRACT(data, '$.attacker_id') = ?
6 ORDER BY tick;
7
8 -- Player stats over time
```

```
9 SELECT
10     strftime('%Y-%W', d.played_at) as week,
11     AVG(pms.rating) as avg_rating
12 FROM player_match_stats pms
13 JOIN demos d ON pms.demo_id = d.id
14 WHERE pms.player_id = ?
15 GROUP BY week;
```

Performance Considerations

Lazy Loading Positions loaded on demand

Batch Inserts Events inserted in batches of 1000

Index Strategy Indexes on common query paths

Progress Streaming Incremental updates during parsing

Demo files can exceed 100MB, requiring careful memory management.

Design Patterns

THIS CHAPTER DESCRIBES the design patterns used throughout OutOf5K, with implementation examples specific to this codebase.

Pattern Summary

Pattern	Category	Primary Use
Repository	Structural	Database access
Command	Behavioral	Tauri IPC
Observer	Behavioral	Events, stores
Factory	Creational	Parser creation
Strategy	Behavioral	Analysis algorithms
Adapter	Structural	API clients
Facade	Structural	Python bridge
Builder	Creational	Query construction

Table 4: Design patterns used in OutOf5K

Repository Pattern

The Repository pattern separates business logic from data access:

```
1 pub struct DemoRepository {
2     pool: SqlitePool,
3 }
4
5 impl DemoRepository {
6     pub async fn find_by_id(
7         &self,
8         id: &str
9     ) -> Result<Option<Demo>> {
10         sqlx::query_as!(Demo,
11             "SELECT * FROM demos WHERE id = ?",
12             id
13         )
14         .fetch_optional(&self.pool)
15         .await
16     }
17
18     pub async fn save(&self, demo: &Demo) -> Result<()>
19     {
20         // Insert logic
21     }
```

The Repository pattern abstracts data persistence, providing a collection-like interface for domain objects.

Benefits:

- Testability via mocking
- Database implementation flexibility
- Centralized query logic

Command Pattern

In Tauri, commands encapsulate IPC requests:

```

1 #[tauri::command]
2 pub async fn import_demos(
3     state: State<'_, AppState>,
4     paths: Vec<String>,
5 ) -> Result<ImportResult, CommandError> {
6     state.demo_service
7         .import_demos(&paths)
8         .await
9 }

```

Frontend invocation:

```

1 import { invoke } from '@tauri-apps/api/tauri';
2
3 const result = await invoke('import_demos', {
4     paths: ['/path/to/demo.demo']
5 });

```

*Observer Pattern**Svelte Stores*

Svelte's reactive stores implement the Observer pattern:

```

1 import { writable } from 'svelte/store';
2
3 function createDemoStore() {
4     const { subscribe, update } = writable({
5         demos: [],
6         loading: false,
7     });
8
9     return {
10         subscribe,
11         addDemo(demo) {
12             update(s => ({
13                 ...s,
14                 demos: [demo, ...s.demos]
15             }));
16         },
17     };
18 }
19
20 export const demoStore = createDemoStore();

```


Tauri Events

Backend-to-frontend notifications:

```

1 // Backend
2 app.emit_all("demo:progress", payload);
3
4 // Frontend
5 listen('demo:progress', (event) => {
6     updateProgress(event.payload);
7 });

```

Factory Pattern

Creating the appropriate parser for demo format versions:

```

1 class DemoParserFactory:
2     @staticmethod
3     def create(file_path: Path) -> DemoParser:
4         version = detect_version(file_path)
5
6         if version == DemoVersion.CS2:
7             return CS2DemoParser(file_path)
8         elif version == DemoVersion.CSG0:
9             return LegacyCSG0Parser(file_path)
10        else:
11            raise UnsupportedDemoError()

```

The Factory pattern creates objects without specifying the exact class, useful when creation depends on runtime conditions.

Strategy Pattern

Different analysis algorithms share a common interface:

```

1 class Analyzer(ABC):
2     @abstractmethod
3     def analyze(self, demo: ParsedDemo) -> Result:
4         pass
5
6 class StatsAnalyzer(Analyzer):
7     def analyze(self, demo: ParsedDemo) -> Result:
8         # Calculate statistics
9         pass
10
11 class HighlightAnalyzer(Analyzer):
12     def analyze(self, demo: ParsedDemo) -> Result:
13         # Detect highlights
14         pass
15
16 # Usage
17 pipeline = AnalysisPipeline([
18     StatsAnalyzer(),
19     HighlightAnalyzer(),
20 ])
21 results = pipeline.run(parsed_demo)

```

Adapter Pattern

Adapting external APIs to internal interfaces:

```

1 #[async_trait]
2 pub trait PlayerStatsProvider {
3     async fn get_stats(&self, id: &str)
4         -> Result<PlayerStats>;
5 }
6
7 // HLTv adapter
8 pub struct HLTvAdapter { /* ... */ }
9
10 #[async_trait]
11 impl PlayerStatsProvider for HLTvAdapter {
12     async fn get_stats(&self, id: &str)
13         -> Result<PlayerStats> {
14         let html = self.fetch_page(id).await?;
15         let hltv_stats = parse_html(&html)?;
16
17         // Adapt to internal format
18         Ok(PlayerStats {
19             rating: hltv_stats.rating_2_0,
20             kd_ratio: hltv_stats.kd,
21             // ...
22         })
23     }
24 }

```

Facade Pattern

The Python bridge provides a simplified interface to the parsing subsystem:

```

1 pub struct PythonBridge {
2     process: Child,
3     // ...
4 }
5
6 impl PythonBridge {
7     // High-level API (Facade)
8     pub async fn parse_demo(&mut self, path: &Path)
9         -> Result<ParsedDemo> {
10         self.send_command("parse", json!({
11             "file_path": path
12         })).await
13     }
14
15     pub async fn get_metadata(&mut self, path: &Path)
16         -> Result<DemoMetadata> {
17         self.send_command("get_metadata", json!({
18             "file_path": path
19         })).await
20     }
21 }

```

Builder Pattern

Constructing complex database queries:

```
1 let (query, bindings) = DemoQueryBuilder::new()  
2   .with_status(DemoStatus::Parsed)  
3   .with_map("de_dust2")  
4   .played_after(week_ago)  
5   .order_by_played_at(true)  
6   .limit(20)  
7   .build();
```


Architecture Decisions

THIS CHAPTER DOCUMENTS significant architectural decisions using the Architecture Decision Record (ADR) format. Each decision includes context, rationale, and consequences.

ADR-001: Tauri as Desktop Framework

Context

OutOf5K requires a desktop application framework that supports:

- Local file system access for large demo files
- Modern, responsive UI
- Reasonable bundle size for distribution
- Cross-platform support

Decision

Use **Tauri v2** as the desktop framework.

Consequences

Positive:

- Small bundle size (10–20MB vs Electron’s 150MB+)
- Better performance with system webview
- Security-first permission model
- Rust backend for performance-critical operations

Negative:

- Younger ecosystem than Electron
- Webview behavior varies across platforms
- Rust learning curve

Alternatives Rejected

Electron Large bundle size, higher memory usage

Qt/PyQt Dated look, smaller ecosystem

Flutter Dart unfamiliarity, less mature desktop support

*ADR-002: Svelte as Frontend Framework**Context*

Need a frontend framework for complex UI including data visualizations, interactive maps, and responsive design.

Decision

Use **Svelte 5** with TypeScript.

*Consequences***Positive:**

- Minimal boilerplate compared to React
- Excellent performance (compiles to vanilla JS)
- Built-in stores, transitions, animations
- Strong Tauri integration

Negative:

- Smaller ecosystem than React
- Smaller developer talent pool
- Svelte 5 is relatively new

*ADR-003: Python Subprocess for Demo Parsing**Context*

CS2 demo parsing requires specialized libraries. The best option is **demoparser2**, a Python library.

Decision

Use Python as a **subprocess** communicating via stdin/stdout JSON.

*Consequences***Positive:**

- Access to best demo parser (demoparser2)
- Python ecosystem for future ML features
- Process isolation (parser crashes don't affect main app)
- Independent development and testing

Negative:

- Process spawn overhead
- Distribution complexity (bundle Python or require installation)
- IPC latency

Distribution Strategy

Use PyInstaller to create a standalone Python executable bundled with the app.

ADR-004: SQLite as Primary Database

Context

Need persistent storage for demos, events, statistics, and settings. Desktop app requires zero-configuration setup.

Decision

Use **SQLite** accessed via `sqlx` in Rust.

Consequences

Positive:

- Zero configuration required
- Single file (easy backup/migration)
- Excellent performance for single-user workloads
- Full SQL support including JSON functions

Negative:

- Single-writer limitation
- No built-in replication for team features
- Limited full-text search

ADR-005: PlantUML for Diagrams

Context

Documentation requires architecture and UML diagrams that are version-controlled and LaTeX-compatible.

Decision

Use **PlantUML** for all technical diagrams.

Consequences

Positive:

- Text-based, version controllable
- Excellent C4 and UML support
- LaTeX-compatible outputs (PDF, PNG, SVG)

- Wide tool support (VS Code, IntelliJ, CLI)

Negative:

- Requires Java
- Learning curve for syntax
- No native GitHub rendering (unlike Mermaid)

*ADR-006: Phased 3D Implementation**Context*

Users want 3D replay capability, but building a full 3D viewer is complex and time-consuming.

Decision

Implement visualization in phases:

1. Phase 1 (MVP): 2D radar-style map
2. Phase 2: Enhanced 2D with paths, utility, heatmaps
3. Phase 4: 3D replay viewer (scope TBD)

*Consequences***Positive:**

- Ship faster with 2D MVP
- Validate core features before 3D investment
- User feedback informs 3D priorities
- 2D is still valuable for analysis

Negative:

- Not full-featured initially
- Less exciting for marketing

Success Criteria

Proceed to 3D only when:

- Users explicitly request 3D features
- Technical feasibility is proven
- Resources are available

Development

THIS CHAPTER COVERS the practical aspects of developing OutOf5K, including setup, workflow, and contribution guidelines.

Prerequisites

Software	Version	Purpose
Node.js	20.x LTS	Frontend
Rust	1.75+	Tauri backend
Python	3.11+	Demo parsing
pnpm	8.x	Package management

Table 5: Required software

Quick Start

```
1 # Clone repository
2 git clone https://github.com/your-org/outof5k.git
3 cd outof5k
4
5 # Install dependencies
6 pnpm install
7
8 # Set up Python
9 cd python
10 python3.11 -m venv venv
11 source venv/bin/activate
12 pip install -r requirements.txt
13 cd ..
14
15 # Run development server
16 pnpm tauri dev
```

Project Structure

src/ Svelte frontend

src-tauri/ Rust/Tauri backend

python/ Python demo parser

docs/ Documentation

The codebase is organized into three main areas: frontend (Svelte), backend (Rust), and parser (Python).

Frontend Structure

```

1 src/|—
2   lib/|
3     |— components/    # UI components|
4     |— stores/        # Svelte stores|
5     |— api/           # Tauri IPC wrappers|
6     |— utils/         # Utilities|—
7   routes/            # SvelteKit pages

```

Backend Structure

```

1 src-tauri/src/|—
2   commands/      # IPC handlers|—
3   services/      # Business logic|—
4   db/            # Database layer|—
5   models/        # Domain models|—
6   api/           # External clients

```

Development Workflow

Branch Strategy

- **main**: Production-ready code
- **develop**: Integration branch
- **feature/***: New features
- **fix/***: Bug fixes

Commit Messages

Follow Conventional Commits:

```

1 feat(parser): add CS2 demo format v4 support
2 fix(ui): handle empty demo list
3 docs(readme): update installation steps

```

Testing

Frontend

```

1 pnpm test          # Run tests
2 pnpm test:coverage # With coverage
3 pnpm test:watch    # Watch mode

```

Rust

```

1 cd src-tauri
2 cargo test          # All tests
3 cargo test -- --nocapture # With output

```

Python

```
1 cd python
2 pytest                # All tests
3 pytest --cov          # With coverage
```

Code Quality

Linting

```
1 # Frontend
2 pnpm lint
3
4 # Rust
5 cargo clippy
6
7 # Python
8 ruff check .
```

Formatting

```
1 # Frontend
2 pnpm format
3
4 # Rust
5 cargo fmt
6
7 # Python
8 black .
```

Building

Development

```
1 pnpm tauri dev
```

Production

```
1 pnpm tauri build
2
3 # Outputs:
4 # macOS: src-tauri/target/release/bundle/dmg/
5 # Linux: src-tauri/target/release/bundle/deb/
6 # Windows: src-tauri/target/release/bundle/msi/
```

Contributing

Process

1. Fork the repository

2. Create a feature branch
3. Make changes with tests
4. Submit a pull request
5. Address review feedback

Code Review Criteria

- Correctness: Does it work?
- Tests: Are there adequate tests?
- Style: Does it follow guidelines?
- Performance: Any concerns?
- Documentation: Is it documented?

Debugging

Frontend

Open DevTools with `Cmd+Option+I` (Mac) or `Ctrl+Shift+I` (Windows/Linux).

Rust

```
1 RUST_BACKTRACE=1 cargo run
```

Database

```
1 sqlite3 ~/Library/Application\ Support/\
2   com.outof5k.app/outof5k.db
3
4 .tables
5 .schema demos
6 SELECT COUNT(*) FROM demos;
```

Database Schema

Core Tables

demos

Stores parsed demo file metadata.

```
1 CREATE TABLE demos (  
2     id TEXT PRIMARY KEY,  
3     file_path TEXT NOT NULL UNIQUE,  
4     file_hash TEXT NOT NULL,  
5     map_name TEXT NOT NULL,  
6     played_at DATETIME,  
7     duration_ticks INTEGER NOT NULL,  
8     tickrate INTEGER NOT NULL DEFAULT 64,  
9     status TEXT NOT NULL DEFAULT 'pending',  
10    parsed_at DATETIME,  
11    metadata JSON,  
12    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
13    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP  
14 );
```

players

Cached player information across demos.

```
1 CREATE TABLE players (  
2     steam_id TEXT PRIMARY KEY,  
3     name TEXT NOT NULL,  
4     avatar_url TEXT,  
5     is_local_user BOOLEAN DEFAULT FALSE,  
6     first_seen DATETIME DEFAULT CURRENT_TIMESTAMP,  
7     last_seen DATETIME DEFAULT CURRENT_TIMESTAMP  
8 );
```

rounds

Round information for each demo.

```
1 CREATE TABLE rounds (  
2     id INTEGER PRIMARY KEY AUTOINCREMENT,  
3     demo_id TEXT NOT NULL REFERENCES demos(id),  
4     round_number INTEGER NOT NULL,  
5     winner TEXT NOT NULL,  
6     win_reason TEXT NOT NULL,
```

```

7   start_tick INTEGER NOT NULL,
8   end_tick INTEGER NOT NULL,
9   ct_score INTEGER NOT NULL,
10  t_score INTEGER NOT NULL,
11  UNIQUE(demo_id, round_number)
12 );

```

events

Game events with JSON payload.

```

1 CREATE TABLE events (
2   id INTEGER PRIMARY KEY AUTOINCREMENT,
3   demo_id TEXT NOT NULL REFERENCES demos(id),
4   round_id INTEGER NOT NULL REFERENCES rounds(id),
5   tick INTEGER NOT NULL,
6   event_type TEXT NOT NULL,
7   data JSON NOT NULL
8 );

```

Statistics Tables

player_round_stats

Per-round statistics for each player.

```

1 CREATE TABLE player_round_stats (
2   id INTEGER PRIMARY KEY AUTOINCREMENT,
3   demo_id TEXT NOT NULL,
4   round_id INTEGER NOT NULL REFERENCES rounds(id),
5   player_id TEXT NOT NULL REFERENCES players(steam_id)
6   ,
7   kills INTEGER DEFAULT 0,
8   deaths INTEGER DEFAULT 0,
9   assists INTEGER DEFAULT 0,
10  damage INTEGER DEFAULT 0,
11  headshots INTEGER DEFAULT 0,
12  flash_assists INTEGER DEFAULT 0,
13  UNIQUE(demo_id, round_id, player_id)
14 );

```

player_match_stats

Aggregated match statistics.

```

1 CREATE TABLE player_match_stats (
2   id INTEGER PRIMARY KEY AUTOINCREMENT,
3   demo_id TEXT NOT NULL REFERENCES demos(id),
4   player_id TEXT NOT NULL REFERENCES players(steam_id)
5   ,
6   kills INTEGER DEFAULT 0,
7   deaths INTEGER DEFAULT 0,
8   assists INTEGER DEFAULT 0,
9   total_damage INTEGER DEFAULT 0,
10  headshots INTEGER DEFAULT 0,

```

```

10     adr REAL,
11     kast REAL,
12     rating REAL,
13     first_kills INTEGER DEFAULT 0,
14     first_deaths INTEGER DEFAULT 0,
15     clutches_won INTEGER DEFAULT 0,
16     clutches_lost INTEGER DEFAULT 0,
17     UNIQUE(demo_id, player_id)
18 );

```

Indexes

Performance-critical indexes:

```

1 CREATE INDEX idx_demos_map ON demos(map_name);
2 CREATE INDEX idx_demos_played_at ON demos(played_at DESC
3 );
4 CREATE INDEX idx_demos_status ON demos(status);
5
6 CREATE INDEX idx_rounds_demo ON rounds(demo_id);
7
8 CREATE INDEX idx_events_demo_type
9     ON events(demo_id, event_type);
10 CREATE INDEX idx_events_round ON events(round_id);
11 CREATE INDEX idx_events_tick ON events(demo_id, tick);
12
13 CREATE INDEX idx_player_round_stats_demo
14     ON player_round_stats(demo_id);
15 CREATE INDEX idx_player_round_stats_player
16     ON player_round_stats(player_id);
17
18 CREATE INDEX idx_player_match_stats_player
19     ON player_match_stats(player_id);

```

Event Data Examples

Kill Event

```

1 {
2   "attacker_id": "76561198012345678",
3   "victim_id": "76561198087654321",
4   "assister_id": null,
5   "weapon": "ak47",
6   "headshot": true,
7   "penetrated": false,
8   "noscope": false,
9   "through_smoke": false,
10  "attacker_blind": false,
11  "attacker_position": {"x": 1024, "y": -512, "z": 64},
12  "victim_position": {"x": 800, "y": -400, "z": 64}
13 }

```

Grenade Event

```
1 {  
2   "player_id": "76561198012345678",  
3   "grenade_type": "flashbang",  
4   "throw_position": {"x": 100, "y": 200, "z": 64},  
5   "land_position": {"x": 500, "y": 300, "z": 64},  
6   "players_flashed": [  
7     {"player_id": "76561198087654321", "duration": 2.5}  
8   ]  
9 }
```